

MARS ROVER MANIPAL
RESEARCH AI TASKPHASE
Report 10

Adarsh Inaganti
240905294
adarshinaganti006@gmail.com

1. Review questions

1.1 What is a dendrogram? What does it represent?

A dendrogram is a tree-like diagram used mainly in hierarchical clustering to visualize how data points are grouped together. It represents the process and result of hierarchical clustering by showing:

- **Data points and clusters**
 - **Leaves (bottom nodes):** individual data points
 - **Branches:** clusters formed by merging data points or smaller clusters
- **Order of merging**
 - It shows which points/clusters are merged first and which are merged later.
 - **Early merges:** very similar points
 - **Late merges:** less similar clusters
- **Distance/dissimilarity**
 - The height of a merge (y-axis) represents the distance (or dissimilarity) at which two clusters are joined.
 - **Low height:** more similar
 - **More height:** less similar
 - This distance depends on the linkage method.
- **Cluster structure at different levels**

By drawing a horizontal cut line, we can decide:

 - how many clusters to form
 - which data points belong to which cluster

1.2 What is Ward's linkage? Where is it used?

- Ward's linkage is a linkage method used in hierarchical (agglomerative) clustering that merges clusters in a way that causes the minimum increase in within-cluster variance.
- Ward's method minimizes the **inertia**.
- The pair of clusters with the smallest increase in inertia after merging two clusters is merged.
- It is used in hierarchical clustering.
- Ward's linkage is used when
 - data is numerical
 - features are scaled

1.3 What is K-Means++?

K-Means++ is an improved initialization method for the K-Means clustering algorithm.

Its goal is to choose better initial centroids so that K-Means:

- converges faster
- avoids poor local minima
- produces more stable, better clusters

K-Means++ is used in

- K-means clustering
- Large datasets
- Situations where clustering quality and stability matter

1.4 Why do we also need F1 score or precision or recall? Why is accuracy alone not sufficient as a metric?

Accuracy can be high even if the model is unoptimized.

Example

- Suppose 95% of samples are negative.
- A model which predicts always negative gets:
 - Accuracy = 95%
 - Precision = 0, recall = 0
- Accuracy hides the failure.

Accuracy treats all mistakes equally.

Example

- Missing a cancer patient (FN) is worse than a false alarm (FP).
- Spam filters: FP (important mail goes to spam) is worse than FN.
- Precision and recall fix these issues by measuring which type of error matters.

Precision and recall both focus on positive class performances.

Example

- Accuracy averages performance across both classes.
- Precision/recall zoom in on the positive class.
- This is critical in fraud detection, medicine, intrusion detection, etc.

F1 score balances precision and recall

- This is because F1 is the harmonic mean of precision and recall.
- Useful when:
 - Classes are imbalanced
 - Both FP and FN matter

1.5 What are the different metrics of an unsupervised learning model?

Internal validation metrics

- **Silhouette score**

Measures how well a point fits within its cluster vs other clusters.

$$S = \frac{b-a}{\max(a,b)}$$

- **a:** average distance to points in same cluster
- **b:** average distance to points in nearest other cluster
- **Range:** -1 to 1

- **Davies–Bouldin Index (DBI)**

Measures average similarity between each cluster and its most similar cluster.

- A lower value indicates better clustering.
- DBI penalizes overlapping clusters.

- **Calinski–Harabasz Index (CHI)**

Ratio of between-cluster dispersion to within-cluster dispersion.

- Higher value = better
- Works well for compact, well-separated clusters

Distance/compactness metrics

- **Within-Cluster Sum of Squares (WCSS)/Inertia**

$$\sum ||x - \mu||^2$$

- Used in K-Means.
- A lower value indicates tighter clusters.
- Used in the elbow method.
- Not a standalone metric as it always decreases when k increases.

External validation metrics

- **Adjusted Rand Index (ARI)**

Measures similarity between predicted clusters and true labels.

- **Range:** -1 to 1
- 1 indicates a perfect match.

- **Normalized Mutual Information (NMI)**

Measures shared information between clustering and ground truth.

- **Range:** 0 to 1
- It is scale-invariant.

- **Purity**

- Measures dominance of a single class in each cluster.
- Easy to understand but biased toward many clusters.

2. Forward propagation in Neural Networks

2.1 Introduction to Neural Networks

2.1.1 What is a Neural Network?

A neural network is a machine learning model inspired by the human brain. It is composed of interconnected units called neurons, arranged in layers, which learn to model complex relationships between inputs and outputs.

2.1.2 Structure of a neural network

- **Input layer:** Receives raw input features
- **Hidden layer(s):** Performs intermediate computations
- **Output layer:** Produces the final prediction

Each neuron performs a weighted computation followed by a non-linear transformation.

The operations of a neural network can be broadly divided into two phases:

- Forward propagation
- Backpropagation

2.2 Introduction to forward propagation

Forward propagation is the fundamental operational mechanism of a neural network. It defines how input data is transformed into an output prediction using the network's current parameters (weights and biases).

Forward propagation does not involve learning. Instead, it is a deterministic computation that applies a sequence of mathematical transformations to the input data.

2.2.1 Characteristics

- Information flows in one direction (forward only)
- No weight or bias updates occur
- Uses existing parameters of the network
- Executed during both training and inference

2.2.2 Purpose

The primary objectives of forward propagation are:

- To transform raw input features into a meaningful output
- To compute the predicted value $\hat{y}$
- To generate intermediate activations required for loss computation
- To store values that are later used in backpropagation

Without forward propagation,

- Predictions cannot be made
- Loss cannot be calculated
- Model training cannot occur

2.3 Structure of Neural Network in forward propagation

A neural network used in forward propagation consists of

- Input layer
- One or more hidden layers
- Output layer

Each layer (except the input layer) performs two operations

- Linear transformation
- Non-linear activation

Information flows strictly in a forward direction.

Input → Hidden layer(s) → Output

2.3.1 Input layer and feature representation

$$X = [x_1, x_2, x_3, \dots, x_n]^T$$

Here, X is the feature vector.

- Each x_i represents a feature.
- No computation is performed in the input layer.
- The input layer simply passes values to the first hidden layer.

$$a^{(0)} = X$$

2.4 Linear transformation (weighted sum)

The first computational step in any hidden or output layer is the linear transformation.

For layer l ,

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$

- $W^{(l)}$: weight matrix of shape (n_l, n_{l-1})
- $a^{(l-1)}$: activations from previous layer
- $b^{(l)}$: bias vector of shape $(n_l, 1)$
- $z^{(l)}$: pre-activation vector

2.4.1 Interpretation of weights

Each weight represents the importance of an input feature.

- A larger magnitude implies a stronger influence.
- A positive weight implies a positive contribution.
- A negative weight implies an inhibitory effect.

2.4.2 Role of bias

- Bias shifts the activation threshold
- Allows neuron activation even when inputs are zero
- Prevents forced origin constraints

2.5 Neuron-level view

For a single neuron,

$$z = b + w_1x_1 + w_2x_2 + \dots w_nx_n$$

This can be viewed as

- A scoring function.
- A similarity measure.
- A decision boundary in a linear space.

However, the computation is **purely linear**.

2.6 Activation functions

If no activation function is applied,

- The entire network collapses into a single linear function
- Multiple layers provide no benefit
- The model cannot learn non-linear relationships

Therefore, a non-linear activation function is applied.

$$a^{(l)} = f(z^{(l)})$$

Activation functions control

- Whether a neuron fires
- How strongly it activates
- What information passes to the next layer

2.6.1 Sigmoid

$$f(z) = \frac{1}{1+e^{-z}}$$

Properties

- **Output range:** (0, 1)
- Smooth and differentiable
- Interpretable as probability

Limitations

- Saturates for a large value of $|z|$
- Causes vanishing gradients
- Rarely used in hidden layers

A sigmoid function is mainly used in binary classification output layers.

2.6.2 ReLU (Rectified Linear Unit)

$$f(z) = \max(0, z)$$

Properties

- **Output range:** $[0, \infty)$
- Sparse activations
- Computationally efficient

Advantages

- Reduces vanishing gradient problem
- Enables deeper networks

Limitations

- Dying ReLU problem (neurons stuck at zero)

A ReLU function is the most commonly used activation in hidden layers.

2.6.3 tanh

$$f(z) = \tanh(z)$$

Properties

- **Output range:** $(-1, 1)$
- Zero-centered
- Stronger gradients than sigmoid

Limitations

- Still suffers from saturation at extremes.

2.6.4 Softmax

$$f(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Properties

- Converts raw scores into probabilities
- Probabilities sum to 1
- Emphasizes the largest scores

Softmax is used exclusively in multi-class classification output layers.

2.7 Layer-by-layer forward propagation

Forward propagation applies the same two steps repeatedly:

- Linear transformation
- Activation

For a network with L layers,

$$\begin{aligned}z^{(1)} &= W^{(1)}X + b^{(1)} \\a^{(1)} &= f^{(1)}(z^{(1)}) \\z^{(2)} &= W^{(2)}a^{(1)} + b^{(2)} \\a^{(2)} &= f^{(2)}(z^{(2)}) \\&\vdots \\z^{(L)} &= W^{(L)}a^{(L-1)} + b^{(L)} \\a^{(L)} &= f^{(L)}(z^{(L)})\end{aligned}$$

The final output is

$$\hat{y} = a^{(L)}$$

2.8 Output Layer and Task Dependency

The output layer differs based on the problem type:

2.8.1 Regression

- **Activation:** Linear
- **Output:** Continuous value

2.8.2 Binary classification

- **Activation:** Sigmoid
- **Output:** Probability of positive class

2.8.3 Multi-class classification

- **Activation:** Softmax
- **Output:** Class probability distribution

2.9 Intuition behind forward propagation

Forward propagation can be interpreted as

- A sequence of feature transformations
- Each layer refines the representation
- Earlier layers learn simple patterns
- Deeper layers learn abstract concepts

Example intuition

- **First layer:** edges or thresholds
- **Middle layers:** shapes or combinations
- **Final layer:** class-specific features

2.10 Role of forward propagation during training

During training,

- Forward propagation computes predictions
- Loss is calculated using predicted output
- Intermediate values are stored
- Backpropagation uses these values to compute gradients

Thus, forward propagation is a prerequisite for learning.

2.11 Limitations of forward propagation alone

Forward propagation by itself

- Does not update parameters
- Does not reduce error
- Depends entirely on initial weights

Learning occurs only when combined with

- Loss functions
- Backpropagation
- Optimization algorithms

3. Neuron computation

3.1 Introduction

3.1.1 What is a neuron?

A neuron is the basic computational unit of a neural network. During forward propagation, each neuron takes multiple inputs, processes them using weights and bias, applies an activation function, and produces an output.

3.1.2 What is neuron computation?

Neuron computation explains how a single neuron transforms input values into an activation, which later contributes to the final prediction.

3.2 Inputs to a neuron

Each neuron receives

- Inputs from the previous layer
- These inputs may be:
 - Raw features (for the first hidden layer)
 - Activations from earlier neurons (for deeper layers)

Let the input vector be

$$x = [x_1, x_2, x_3, \dots, x_n]^T$$

Here, x_i represents the i^{th} input feature/activation.

3.3 Weights associated with inputs

Each input has an associated weight.

$$w = [w_1, w_2, w_3, \dots, w_n]^T$$

3.3.1 Interpretation of weights

The weight of an input represents its importance.

- A larger magnitude implies a stronger influence.
- A positive weight implies an increased neuron output.
- A negative weight implies a decreased neuron output.

Weights are learned during training but remain fixed during forward propagation.

3.4 Weighted sum (linear combination)

The first computation performed by a neuron is a linear combination of inputs and weights, followed by addition of a bias.

$$z = \sum_{i=1}^n w_i x_i + b$$

In the vector form,

$$z = w^T x + b$$

Here, z is the pre-activation value and b is the bias term.

3.5 Role of bias in neuron computation

Bias allows the neuron to

- Shift the activation function left or right
- Activate even when all inputs are zero
- Learn patterns that do not pass through the origin

Without bias,

- Neuron output would be strictly dependent on input magnitude
- Model flexibility would be reduced
- Bias acts as a trainable offset in neuron computation.

3.6 Interpretation of the linear step

The weighted sum z can be interpreted as

- A scoring function
- A measure of similarity between inputs and weights
- A decision boundary in feature space

However,

- This step alone is purely linear
- Linear models cannot represent complex real-world patterns
- This motivates the next step: activation.

3.7 Application of activation functions

The neuron output is obtained by applying an activation function $f(\cdot)$ to the pre-activation value.

$$a = f(z)$$

Here, a is the neuron's activation and f is a non-linear function.

3.7.1 Purpose

- Introduces non-linearity
- Controls neuron firing
- Determines information passed to the next layer

3.8 Geometric interpretation

From a geometric perspective,

- The weighted sum defines a hyperplane
- Bias shifts the hyperplane
- Activation function shapes the response around the boundary

Thus, a neuron can be viewed as

- A soft decision unit
- A feature detector

3.9 Neuron computation in multi-layer networks

In deeper networks

- Each neuron operates on activations, not raw inputs
- Neurons in higher layers learn more abstract representations
- The same computation formula applies at every layer

This uniform structure allows neural networks to scale efficiently.

3.10 Importance of neuron computation

Neuron computation is fundamental because

- It is the smallest unit of learning
- Entire networks are built by stacking neurons
- Errors and gradients are computed at neuron level

3.11 Limitations of a single neuron

A single neuron

- Can only model linear decision boundaries
- Cannot capture complex non-linear patterns alone

Complex representations emerge only when multiple neurons, multiple layers and non-linear activations are combined.